EXPERIMENT NO: 03

Aim: Implement at a program for Group Communication

Theory:

What is Group Communication in Distributed Systems?

Group communication in distributed systems refers to the process where multiple nodes or entities communicate with each other as a group.

- Instead of sending messages to individual recipients, group communication allows a sender to transmit information to all members of a group simultaneously.
- This method is essential for coordinating actions, sharing data, and ensuring that all participants in the system are informed and synchronized. It's particularly useful in scenarios like collaborative applications and real-time updates.

Importance of Group Communication in Distributed Systems

Group communication is critically important in distributed systems due to several key reasons:

- Multiple nodes must collaborate and synchronize their actions. Group communication helps them exchange information and stay updated.
- Different nodes can create data that needs to be shared. Group communication helps quickly send this information to everyone involved, reducing delays and keeping data
- consistent. Group communication protocols enhance reliability by allowing messages to be replicated or acknowledged across multiple nodes. This ensures robust communication, even during failures or network issues.
- As distributed systems expand, effective scaling is crucial. Group communication mechanisms can manage more nodes and messages without sacrificing performance, keeping the system efficient and responsive.

Types of Group Communication in a Distributed System

Below are the three types of group communication in distributed systems:

1. Unicast Communication

Unicast communication is the point-to-point transmission of data between two nodes in a network. In the context of distributed systems:

- Unicast is when a sender sends a message to a specific recipient, using their unique network address.
- Each message targets one recipient, creating a direct connection between the sender and the receiver.
- You commonly see unicast in client-server setups, where a client makes requests and receives responses, as well as in direct connections between peers.
- This method makes good use of network resources, is easy to implement, and keeps latency low because messages go straight to the right person.
- Unicast isn't efficient for sending messages to many recipients at once, as it requires separate messages for each one, leading to more work.

2. Multicast Communication

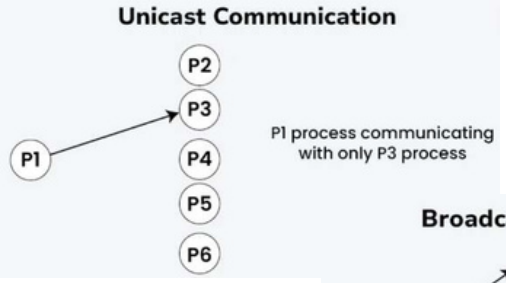
Multicast communication involves sending a single message from one sender to multiple receivers simultaneously within a network. It is particularly useful in distributed systems where broadcasting information to a group of nodes is necessary:

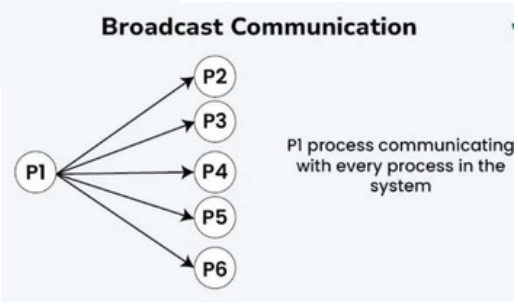
- Multicast lets a sender share a message with a specific group of people who want it.
- This way, the sender can reach many people at once, which is more efficient than sending separate messages.
- This approach is often used to send updates to subscribers or in collaborative applications where real-time sharing of changes is needed.
- By sending data just once to a group, multicast saves bandwidth, simplifies communication, and can easily handle a larger number of recipients.
- Managing group membership is necessary to ensure reliable message delivery, and multicast can run into issues if there are network problems that affect everyone in the group.

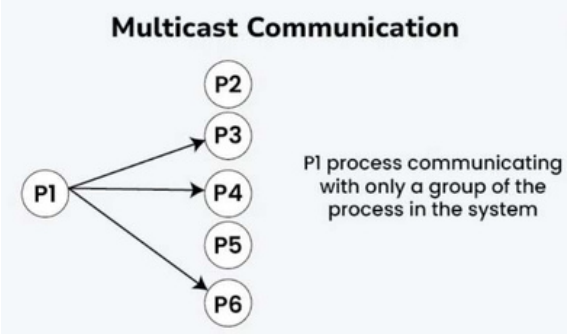
3. Broadcast Communication

Broadcast communication involves sending a message from one sender to all nodes in the network, ensuring that every node receives the message:

- Broadcast is when a sender sends a message to every node in the network without targeting specific recipients.
- Messages are delivered to all nodes at once using a special address designed for this purpose.
- It's often used for network management tasks, like sending status updates, or for emergency alerts that need to reach everyone quickly.
- Broadcast ensures that every node receives the message without needing to specify who the recipients are, making it efficient for sharing information widely.
- It can cause network congestion in larger networks and raises security concerns since anyone on the network can access the broadcast message, which might lead to unauthorized access.





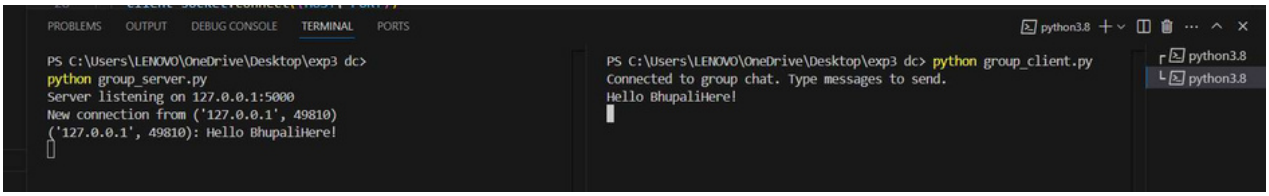


Server Code (group_server.py)

```
import socket
import threading

# Server Configuration
HOST = '127.0.0.1'  # Localhost
PORT = 5000
clients = []  # List to store connected clients

# Function to handle each client
def handle_client(client_socket, address):
    print(f"New connection from {address}")
    while True:
        try:
            message = client_socket.recv(1024).decode('utf-8')
            if not message:
                break
            print(f"{address}: {message}")
            broadcast(message, client_socket)
        except:
            clients.remove(client_socket)
            client_socket.close()
            break

# Function to broadcast messages to all clients
def broadcast(message, sender_socket):
    for client in clients:
        if client != sender_socket:
            try:
                client.send(message.encode('utf-8'))
            except:
                clients.remove(client)

# Main function to start the server
def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(5)
    print(f"Server listening on {HOST}:{PORT}")

    while True:
        client_socket, client_address = server.accept()
        clients.append(client_socket)
        thread = threading.Thread(target=handle_client, args=(client_socket, client_address))
        thread.start()

if __name__ == "__main__":
    start_server()
```

Client Code (group_client.py)

```
import socket
import threading

# Client Configuration
HOST = '127.0.0.1'
PORT = 5000

# Function to receive messages from the server
def receive_messages(client_socket):
    while True:
        try:
            message = client_socket.recv(1024).decode('utf-8')
            print("\n" + message)
        except:
            print("Disconnected from server.")
            client_socket.close()
            break

# Function to send messages to the server
def send_messages(client_socket):
    while True:
        message = input()
        client_socket.send(message.encode('utf-8'))

# Main function to start the client
def start_client():
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_socket.connect((HOST, PORT))

    print("Connected to group chat. Type messages to send.")

    receive_thread = threading.Thread(target=receive_messages, args=(client_socket,))
    send_thread = threading.Thread(target=send_messages, args=(client_socket,))

    receive_thread.start()
    send_thread.start()

if __name__ == "__main__":
    start_client()
```



Conclusion: The experiment implemented a real-time group communication system using Python sockets and threading, allowing multiple clients to exchange messages efficiently.

server .py

```
import socket
import threading

# Server Configuration
HOST = '127.0.0.1' # Localhost
PORT = 5001       # Port to listen on
clients = []      # List to store connected client sockets

def broadcast(message, sender_socket):
    """Sends a message to all clients except the sender."""
    for client in clients:
        if client != sender_socket:
            try:
                client.send(message.encode("utf-8"))
            except:
                # If sending fails, the client might be disconnected
                clients.remove(client)

def handle_client(client_socket, address):
    """Handles the lifecycle of a single client connection."""
    print(f"New connection from {address}")
    while True:
        try:
            message = client_socket.recv(1024).decode("utf-8")
            if not message:
                break

            print(f"{address}: {message}")
            broadcast(message, client_socket)
        except:
            break

    # Cleanup when client disconnects
    if client_socket in clients:
        clients.remove(client_socket)
    client_socket.close()
    print(f"Connection closed for {address}")

def start_server():
    """Initializes the server and listens for incoming connections."""
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(5)
    print(f"Server listening on {HOST}:{PORT}")

    while True:
        client_socket, client_address = server.accept()
        clients.append(client_socket)

        # Create a new thread for every new client
        thread = threading.Thread(target=handle_client, args=(client_socket, client_address))
        thread.start()

if __name__ == "__main__":
    start_server()
```

```python
import socket
import threading

# Client Configuration
HOST = '127.0.0.1'
PORT = 5001  # Must match the Server's port

def receive_messages(client_socket):
    """Continuously listens for messages from the server."""
    while True:
        try:
            # Receive and decode the message
            message = client_socket.recv(1024).decode("utf-8")
            if message:
                print(f"\n{message}")
                print("Type a message: ", end="") # Keep the prompt visible
            else:
                break
        except:
            print("\nDisconnected from server.")
            client_socket.close()
            break

def send_messages(client_socket):
    """Continuously waits for user input to send to the server."""
    while True:
        try:
            message = input("Type a message: ")
            if message.lower() == 'quit':
                client_socket.close()
                break
            client_socket.send(message.encode("utf-8"))
        except:
            break

def start_client():
    """Initializes the client and starts communication threads."""
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:
        client_socket.connect((HOST, PORT))
        print("Connected to group chat. Type 'quit' to exit.")
    except ConnectionRefusedError:
        print("Could not connect to server. Is it running?")
        return

    # Create two threads: one for receiving, one for sending
    receive_thread = threading.Thread(target=receive_messages, args=(client_socket,))
    send_thread = threading.Thread(target=send_messages, args=(client_socket,))

    # Start both threads
    receive_thread.start()
    send_thread.start()

if __name__ == "__main__":
    start_client()
```